

TRANSFORMER ATTENTION ACCELERATOR

Full RTL Implementation on Xilinx KV260 FPGA

Platform	Xilinx KV260 FPGA
HDL	SystemVerilog
Tools	Vivado 2023.x
Timeline	4–6 Weeks
Team Size	2 Engineers
Target	NVIDIA / AMD Recruiting

CONFIDENTIAL — INTERNAL PROJECT REFERENCE

1. Project Overview

1.1 What This Project Is

This project implements a complete, synthesizable Transformer Attention Engine in SystemVerilog, deployed on a Xilinx KV260 FPGA. Every computation — from reading input matrices to producing the final output — is performed by custom RTL hardware. There is no CPU doing arithmetic, no GPU, and no software kernels involved.

Think of it this way: you are building a chip (emulated on an FPGA) that can run the attention mechanism from a Transformer model — the mathematical core that makes models like GPT understand context. When the chip is done, you feed it matrices and it gives you back context-aware output vectors, entirely in hardware.

1.2 What Attention Actually Computes

In a Transformer, every token (word or sub-word) in a sequence needs to understand how relevant every other token is to it. This is done with three vectors per token:

Vector	What it means	Analogy
Q (Query)	What is this token looking for?	A search query you type into Google
K (Key)	What information does this token advertise?	The title/tags of a webpage
V (Value)	What information does this token actually hold?	The actual content of the webpage

The computation has four steps:

1. **Score:** Multiply Q by K^T . Every token scores every other token. Result: an $N \times N$ score matrix S .
2. **Scale:** Divide scores by $\sqrt{d_k}$ to prevent values from getting too large. (d_k = dimension of each head)
3. **Normalize:** Apply softmax row-by-row. Converts raw scores into probabilities that sum to 1. Result: attention weight matrix A .
4. **Blend:** Multiply A by V . Each token gets a weighted blend of all value vectors. Result: output matrix O .

One-sentence summary

Given Q , K , V as input matrices, your FPGA computes $O = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$ — entirely in hardware RTL.

1.3 What Your Design Receives and Outputs

Signal	Shape	Description
Q (input)	$N \times d_k$	Query matrix — one row per token

K (input)	$N \times d_k$	Key matrix — one row per token
V (input)	$N \times d_k$	Value matrix — one row per token
O (output)	$N \times d_k$	Output — each row is a context-enriched token vector
N	64 (target)	Sequence length — number of tokens
d_k	64 (target)	Head dimension — vector length per token
Data format	16-bit fixed-point	Q8.8 format (8 integer bits, 8 fractional bits)

Correctness check

Your hardware output O must match a NumPy golden reference (computed in float32) within a max absolute error of 0.01 per element. This is your pass/fail correctness criterion.

1.4 What BRAM Is

BRAM stands for Block RAM. It is memory that lives inside the FPGA chip itself — not external RAM, not off-chip DDR. Think of it as the FPGA's private scratchpad sitting physically next to your logic, so reads and writes happen in 1–2 clock cycles.

You use BRAM to store Q, K, V before computation starts, to store the intermediate score matrix S, and to write the output O when done. The KV260 has 4.5 MB of BRAM — your 64×64 design needs about 32 KB, which is less than 1% of available BRAM.

2. Hardware Architecture

2.1 Pipeline Overview

The design is a four-stage hardware pipeline. Data flows from BRAM through each stage sequentially, controlled by a top-level FSM. All stages share a single clock domain targeting 100–200 MHz.

Stage	Module Name	What it does
①Load	bram_if	Reads Q, K, V from on-chip BRAM into the compute pipeline
②Score	qk_systolic	Computes $Q \times K^T$ using a systolic PE array — raw attention scores S
③Scale + Normalize	softmax_unit	Divides by $\sqrt{d_k}$, finds row max, computes $\exp()$ via LUT, normalizes — outputs weights A
④Blend	av_multiply	Computes $A \times V$ using a second accumulator array — outputs context vectors O
Control	top_fsm	Sequences all stages with valid/ready handshake signals

2.2 Module Breakdown and Ownership

Module	Owner	Details
qk_systolic.sv	You	Systolic PE mesh computing $Q \times K^T$. Reuse and scale your prior systolic array. Each PE does one 16-bit multiply-accumulate per cycle.
av_multiply.sv	You	Accumulator array computing $A \times V$. Similar structure to qk_systolic but reads from the softmax output.
softmax_unit.sv	Teammate	Per-row softmax: subtract row max for numerical stability, look up $\exp()$ in a BRAM LUT, sum and normalize with fixed-point reciprocal.
bram_if.sv	Teammate	AXI4-Lite or direct-port BRAM controller. Loads Q/K/V before compute, ping-pong buffers V rows, writes O after compute.
top_fsm.sv	Shared	Master FSM wiring all modules. Defines valid/ready handshakes. This is your main integration surface.

2.3 Fixed-Point Number Format

FPGAs do not have floating-point units by default — they are expensive. Instead, you use fixed-point arithmetic throughout.

Format: Q8.8 (16 bits total)

8 bits represent the integer part. 8 bits represent the fractional part. The decimal point is always in the same position. To store 1.5, you store the integer 384 ($= 1.5 \times 256$). All your MAC units operate on plain integers — you just track where the decimal point is.

Key rule: after every multiply, you accumulate 32-bit intermediate results internally, then truncate back to 16-bit at the output of each stage. Choosing where to truncate (rounding strategy) directly affects your error vs. the float reference.

2.4 Softmax Implementation Strategy

Softmax is the hardest module. You cannot call `exp()` in RTL. The standard hardware approach has three steps:

5. Find the row maximum: scan all N scores in the row, register the max value.
6. Subtract and look up: compute $(\text{score} - \text{max})$ for each element, use it as an index into a pre-computed BRAM LUT of `exp()` values. The subtraction before `exp()` prevents overflow — this is called the numerically stable softmax trick.
7. Normalize: sum all `exp()` outputs, compute the reciprocal (using a shift-and-add or Newton-Raphson approximation), multiply each `exp()` value by the reciprocal.

Why subtract the max first?

`exp(large number)` overflows any fixed-point register. By subtracting the row max, all inputs to `exp()` are ≤ 0 , so `exp()` returns values in $(0, 1]$. This is mathematically equivalent but numerically safe. This trick is used in every production attention kernel.

3. Input / Output Specification

3.1 Target Configuration

Parameter	Value	Notes
Sequence length N	64	Number of tokens in the input sequence
Head dimension d _k	64	Length of each token's Q/K/V vector
Score matrix S	64 × 64 = 4,096 values	Output of the QK stage
Data width	16-bit fixed-point (Q8.8)	All I/O and internal datapaths
BRAM for Q + K + V	≈ 24 KB	3 matrices × 64 × 64 × 2 bytes
BRAM for scores S	≈ 8 KB	64 × 64 × 2 bytes
Total BRAM needed	≈ 32 KB	< 1% of KV260's 4.5 MB
Clock target	100–200 MHz	Synthesis target on KV260
Latency target	< 10,000 cycles	For one full attention pass at N=64

3.2 RTL Port Interface (top_fsm.sv)

Signal	Description
clk	System clock, single domain
rst_n	Active-low synchronous reset
start	Pulse high for 1 cycle to begin computation
q_data [15:0]	Q matrix element input (streamed row by row)
k_data [15:0]	K matrix element input
v_data [15:0]	V matrix element input
data_valid	High when input data is valid
out_data [15:0]	Output O element (streamed row by row)
out_valid	High when output data is valid
done	Pulses high when full output matrix O is complete

3.3 What the Output Should Look Like

The output is an $N \times d_k$ matrix O, streamed out one element per cycle when out_valid is high. In a 4-word example — 'The cat sat down' — the input is 4 vectors (one per word). The output is also 4 vectors, but now each vector has been enriched with context from all 4 words.

Concrete correctness test

Generate random Q, K, V matrices in Python (numpy). Compute the expected output in float32. Load Q, K, V into your simulation testbench. Run your RTL. Collect out_data values. Compare element-wise: $\max(\text{abs}(\text{rtl_out} - \text{numpy_out}))$ must be < 0.01 .

4. Week-by-Week Plan

Week	You (Compute modules)	Teammate (Memory + Normalize)
Week 1	Scale up systolic PE from prior project. Write <code>qk_systolic.sv</code> . Unit testbench: feed 4×4 Q and K, verify S against hand-calculated values.	Implement <code>bram_if.sv</code> with direct-port BRAM. Write softmax LUT generator (Python script). Unit testbench: verify BRAM read/write.
Week 2	Extend <code>qk_systolic</code> to 64×64. Verify timing with Vivado behavioral sim. Fix any pipeline bubble bugs.	Implement <code>softmax_unit.sv</code> : row max, LUT lookup, reciprocal norm. Unit testbench: compare output against NumPy softmax.
Week 3	Implement <code>av_multiply.sv</code> . Unit testbench: feed known A and V, verify O.	Write <code>top_fsm.sv</code> skeleton. Define handshake interface. Begin integration.
Week 4	Integration: wire all modules through <code>top_fsm</code> . Run end-to-end simulation with 8×8 input. Debug mismatches against NumPy reference.	Integration: same. Focus on BRAM timing — ensure V rows are ready when <code>av_multiply</code> needs them.
Week 5	Synthesize in Vivado. Fix timing violations. Run implementation. Check utilization report (LUT, DSP, BRAM counts).	Load bitstream onto KV260. Drive inputs from Python via UART/AXI. Collect outputs. Run correctness check.
Week 6	Sweep N=16, 64, 128. Measure latency in cycles and real time. Write up architecture section of report.	Sweep 8-bit vs 16-bit fixed-point. Measure error vs. NumPy at each bit width. Write up results/benchmarks section.

5. Benchmarks and Metrics to Report

These are the numbers you will report in your documentation, presentation, and resume. Every number should come from a real Vivado report or simulation measurement — not estimated.

5.1 Correctness Metrics

Metric	How to measure
Max Absolute Error (MAE)	$\max(\text{abs}(\text{rtl_output} - \text{numpy_float32_output}))$ — target < 0.01
Mean Absolute Error	$\text{mean}(\text{abs}(\text{rtl_output} - \text{numpy_float32_output}))$ — target < 0.005
Pass rate	% of test vectors where MAE < 0.01 — target 100% on random inputs
Bit width sensitivity	Report MAE for 8-bit vs 12-bit vs 16-bit fixed-point

5.2 Performance Metrics

Metric	How to measure
Latency (cycles)	Cycles from start pulse to done pulse. Measure in simulation waveform.
Latency (ms)	$\text{Latency (cycles)} \div \text{clock frequency in Hz} \times 1000$
Throughput (tokens/sec)	$N \text{ tokens} \div \text{latency (seconds)}$
Clock frequency (MHz)	From Vivado timing report — Worst Negative Slack (WNS) tells you how much headroom you have
Theoretical peak GOPS	$(2 \times N^2 \times d_k \text{ operations}) \div \text{latency (seconds)} \div 1e9$

5.3 Resource Utilization Metrics

From Vivado Implementation Report (open after running implementation):

Resource	What to record
LUT count	Number of look-up tables used vs. KV260 total (117,120 available)
DSP count	Number of DSP slices used vs. available (1,728). Each MAC in your PE uses 1–2 DSPs.
BRAM count	Number of 36Kb BRAM tiles used vs. available (144)
FF (Flip-Flops)	Register count — indicates pipeline depth
LUT utilization %	$(\text{Used LUTs} \div 117,120) \times 100$
DSP utilization %	$(\text{Used DSPs} \div 1,728) \times 100$

5.4 Design Space Exploration

This is what separates a strong project from an average one. Report how design choices trade off against each other:

Experiment	What to vary and measure
Bit width sweep	Run 8-bit, 12-bit, 16-bit fixed-point. Plot MAE vs. DSP count for each.
Sequence length sweep	Run $N = 16, 32, 64, 128$. Plot latency (cycles) vs. N . Should scale as N^2 .
PE array size	Try $4 \times 4, 8 \times 8, 16 \times 16$ systolic array. Plot latency vs. LUT/DSP count.
Clock frequency	Attempt 100 MHz, 150 MHz, 200 MHz. Report WNS and whether timing closes.

6. Tools and Workflow

Tool	Purpose	When to use it
Vivado (Xilinx)	Main design environment — write, simulate, synthesize, implement	Every day
Vivado Simulator (xsim)	Behavioral and post-synthesis simulation, waveform viewer	Weeks 1–4 (before hardware)
Vivado Synthesis	Converts RTL to gate-level netlist. Produces LUT/DSP/FF counts.	Week 5
Vivado Implementation	Place and route on KV260 fabric. Produces timing and utilization reports.	Week 5
Vivado Timing Report	Shows clock frequency and Worst Negative Slack (WNS). $WNS \geq 0$ means timing closed.	Week 5–6
Python + NumPy	Generate test vectors, compute golden reference, compare against RTL output	Weeks 1–6
SystemVerilog Testbench	Drives your RTL with inputs, captures outputs, auto-checks against expected values	Weeks 1–5
Git	Version control. Commit working checkpoints before making big changes.	Every day

6.1 Vivado Workflow (Step by Step)

8. Create a new RTL project in Vivado. Target board: KV260. Add your .sv source files and testbench files.
9. Run Behavioral Simulation (xsim). Check waveforms. Your signals should match expectations before you synthesize.
10. Run Synthesis (Flow > Run Synthesis). Check the synthesis log for warnings. Open the utilization report to see LUT/DSP/BRAM counts.
11. Run Implementation (Flow > Run Implementation). This does place-and-route. Open the timing report and check WNS — it must be ≥ 0 for your design to work at your target clock.
12. Generate Bitstream and program the KV260. Use Vivado Hardware Manager to flash the FPGA.
13. Validate on hardware: drive inputs from a Python script over UART or AXI, read back outputs, compare to NumPy golden reference.

6.2 Recommended File Structure

File / Folder	Contents
---------------	----------

src/qk_systolic.sv	QK dot-product systolic array module (you)
src/av_multiply.sv	AV multiply accumulator module (you)
src/softmax_unit.sv	Softmax unit with LUT (teammate)
src/bram_if.sv	BRAM interface controller (teammate)
src/top_fsm.sv	Top-level FSM wiring all modules (shared)
tb/tb_qk_systolic.sv	Unit testbench for qk_systolic
tb/tb_softmax.sv	Unit testbench for softmax_unit
tb/tb_top.sv	End-to-end integration testbench
scripts/gen_golden.py	NumPy script: generates Q/K/V, computes expected O, writes to file
scripts/verify.py	Compares RTL output file against NumPy golden, reports MAE
docs/	Architecture diagrams, utilization reports, timing screenshots

7. Interview Questions to Prepare

These are real questions asked by NVIDIA, AMD, Apple Silicon, Qualcomm, and Intel hardware teams. For each, the expected answer is technical and specific to your project — not generic.

7.1 Architecture Questions

Question	What they want to hear
Walk me through your attention accelerator architecture.	Stage-by-stage explanation: BRAM → QK systolic → softmax → AV multiply → output. Mention valid/ready handshake, fixed-point format, clock domain.
Why did you use a systolic array for the matrix multiply?	Regular data flow, no broadcast, high data reuse — each PE passes partial sums to neighbors. Maps well to FPGA fabric. You had prior experience from your previous project.
How does your softmax handle numerical overflow?	Subtract row max before computing $\exp()$. $\exp(x - \max)$ is always in $(0, 1]$. This is the standard numerically stable trick.
Why fixed-point instead of floating-point?	Floating-point requires large IP blocks (DSPs + LUTs). Fixed-point uses fewer resources, runs at higher clock, and for attention with careful bit-width selection, error is acceptable. Show your MAE numbers.
What is Worst Negative Slack and why does it matter?	WNS is how much timing margin you have after place-and-route. $WNS \geq 0$ means your design meets timing at your target clock. Negative WNS = the design will fail on hardware.

7.2 Design Tradeoff Questions

Question	What they want to hear
How would you reduce latency if you needed to?	Increase PE array size (more parallelism), pipeline more stages, increase clock frequency, reduce sequence length. Show your latency-vs-PE-count sweep data.
You used 16-bit fixed-point. What happens if you go to 8-bit?	DSP count drops, clock may go higher, but MAE increases. Reference your bit-width sweep results. There is a Pareto tradeoff between resource use and accuracy.
How does latency scale with sequence length N?	QK multiply is $O(N^2 \cdot d_k)$ — it scales quadratically. At $N=128$ you expect 4× more cycles than $N=64$. Show your sweep data.
How does your design handle pipeline bubbles between stages?	Valid/ready handshake. A stage asserts ready when it can accept data. Upstream holds valid until ready is seen. No data is lost.
If you had more BRAM, what would you change?	Increase N (sequence length), add multi-head support (store multiple K/V sets), or store the full score matrix instead of computing it row-by-row.

7.3 Verification Questions

Question	What they want to hear
How did you verify your hardware is correct?	Compared RTL output against NumPy float32 reference. Measured max absolute error and mean absolute error across random test vectors. Passed if $MAE < 0.01$.
What edge cases did your testbench cover?	All-zero inputs, identity matrices, max-value inputs to check overflow, random matrices for statistical coverage.
What's the difference between functional and timing simulation?	Functional: checks logic is correct, ignores gate delays. Timing: uses actual post-implementation delays to verify design meets timing on real hardware. You should run both.
Did you run any formal verification?	If no: 'We used directed and random testbenches with golden reference comparison. For a full tapeout, we would add formal property checking.' Honest answer is fine.

7.4 FPGA-Specific Questions

Question	What they want to hear
What is a BRAM and why did you use it instead of registers?	BRAM is dedicated on-chip memory blocks in the FPGA fabric. Registers are fast but cost flip-flops. A 64×64 matrix in registers would use 4,096 FFs. BRAM stores the same data in 1 or 2 BRAM tiles. BRAM is the right tool for storing large matrices.
What does 'timing closure' mean?	After place-and-route, all timing paths meet the target clock period. $WNS \geq 0$. If not, you must either reduce clock frequency, add pipeline registers to break long paths, or rethink your datapath.
How many DSP slices does your design use and why?	Answer with your actual number from the Vivado report. Explain: each PE in the systolic array uses 1 DSP for the 16-bit MAC. Total DSPs $\approx$ number of PEs $\times$ 1–2.
What's the difference between LUTs, FFs, and DSPs on an FPGA?	LUTs: implement arbitrary combinational logic (4–6 input truth tables). FFs (flip-flops): store 1-bit state on clock edge — used for pipeline registers. DSPs: hardened multiply-accumulate blocks — faster and more area-efficient than building a multiplier from LUTs.

8. How to Present This on Your Resume

8.1 Resume Bullet Template

Resume bullets (fill in your actual measured numbers)

- Designed and deployed a full RTL Transformer Attention Accelerator in SystemVerilog on a Xilinx KV260 FPGA; 64-token sequence at 64-dim head, achieving [X] MHz clock and [Y] ms end-to-end latency
- Implemented numerically stable fixed-point softmax using BRAM LUT approximation; output matched NumPy float32 reference within [Z] MAE
- Conducted design space exploration across 8/12/16-bit fixed-point formats and systolic array sizes; characterized latency-vs-resource Pareto frontier
- Wrote fully automated SystemVerilog testbench with NumPy golden reference comparison for end-to-end functional verification

8.2 One-Paragraph Portfolio Description

Portfolio / personal website paragraph

Built a synthesizable Transformer Attention Engine in SystemVerilog, deployed on a Xilinx KV260 FPGA. The design implements the full attention computation — QK dot-product via systolic array, numerically stable fixed-point softmax with BRAM LUT approximation, and AV weighted blend — in a pipelined hardware architecture with no CPU or GPU involvement. The design achieved [X] MHz clock frequency and processed a 64-token, 64-dim sequence in [Y] ms. Output correctness was validated against a NumPy float32 golden reference (MAE < 0.01). Design space exploration covered bit-width and PE array size tradeoffs with Vivado utilization and timing reports as evidence.

8.3 What Roles This Targets

Role	Companies	Why this project is relevant
RTL / Digital Design Engineer (New Grad)	NVIDIA, AMD, Intel, Qualcomm, Apple, Broadcom, Arm	Core skill: writing, verifying, and synthesizing RTL datapaths. This is exactly the day-to-day work.
AI Accelerator Architect	NVIDIA, AMD (Instinct), Google (TPU), Tenstorrent, Groq, Cerebras, Graphcore	You understand both the ML workload (attention) and how to build hardware for it. Rare combination in new grads.
FPGA Engineer	Xilinx/AMD, Intel (Altera), defense contractors, HFT firms	End-to-end FPGA flow: RTL → synthesis → implementation → on-board validation.
Hardware Verification Engineer	Same as above	Testbench + golden reference methodology is industry-standard verification practice.

